# REST API - .NET Developer Intern müsahibəsi üçün geniş bələdçi

*Sadə dildən başlayıb praktik səviyyəyə qədər izah*

**Hazırlayan: ChatGPT**

Tarix: 22 aprel 2026

# Mündəricat


- 1. REST API nədir və niyə istifadə olunur?
- 2. HTTP əsasları: request, response, header, body
- 3. HTTP metodları: GET, POST, PUT, PATCH, DELETE
- 4. Status kodları və real müsahibə nüansları
- 5. REST prinsipləri və endpoint dizaynı
- 6. JSON, DTO, Model Binding, Validation
- 7. ASP.NET Core Web API-də əsas anlayışlar
- 8. Authentication və Authorization
- 9. Versioning, pagination, filtering, sorting
- 10. Error handling, logging, Swagger, testing
- 11. Təhlükəsizlik və performans
- 12. Tez-tez verilən müsahibə sualları
- 13. Mini praktik nümunə: Product API
- 14. 1 günlük və 1 həftəlik hazırlıq planı


## 1. REST API nədir və niyə istifadə olunur?

API - Application Programming Interface - bir sistemin başqa sistemlə danışma qaydasıdır. REST API isə HTTP üzərindən çalışan, resurs mərkəzli, sadə və standartlaşdırılmış API yanaşmasıdır.

Məsələn, frontend tətbiqi məhsullar siyahısını serverdən almaq istəyirsə, serverdə belə bir endpoint ola bilər: GET /api/products. Frontend sorğu göndərir, server JSON formatında cavab qaytarır.

### REST API-ni başa düşmək üçün 3 əsas fikir yadında qalsın:

- API iki proqram arasında ünsiyyətdir.
- REST bu ünsiyyəti HTTP qaydaları ilə qurur.
- Əsas fokus əməliyyatdan yox, resursdan gəlir. Məsələn /createProduct yox, /products.

### REST API niyə vacibdir?

- Frontend və backend ayrılığını təmin edir.
- Mobil tətbiq, web tətbiq, üçüncü tərəf servis eyni backend-dən istifadə edə bilər.
- Standart HTTP metodları ilə işlədiyi üçün öyrənilməsi və istifadəsi daha rahatdır.
- Mikroservis arxitekturasında sistemlər arasında əlaqə yaratmaq üçün geniş istifadə olunur.

#### REST API ilə bağlı ən vacib müsahibə cümləsi:

REST API HTTP üzərindən resurslarla işləyən, stateless, vahid interfeysə əsaslanan bir memarlıq üslubudur.

## 2. HTTP əsasları: request, response, header, body

REST API-ni bilmək üçün HTTP-ni yaxşı başa düşmək vacibdir.

### HTTP Request nələrdən ibarətdir?

- Method - nə etmək istəyirik: GET, POST, PUT, PATCH, DELETE və s.
- URL - hansı resursa müraciət edirik.
- Headers - əlavə məlumatlar. Məsələn Content-Type, Authorization.
- Body - göndərilən məlumat. Adətən POST, PUT, PATCH-də olur.

- Query string - filtr, səhifələmə və s. üçün. Məsələn ?page=1&pageSize=10

### HTTP Response nələrdən ibarətdir?

- Status code - nəticə nə oldu.
- Headers - əlavə metadata.
- Body - qaytarılan data və ya error mesajı.

### Sadə nümunə:

Request: GET /api/products/5

Response: 200 OK və body içində {"id":5,"name":"Keyboard","price":45.5}

### Headers haqqında nə bilməlisən?

- Content-Type: body hansı formatdadır. Məsələn application/json
- Accept: client hansı formatda cavab gözləyir.
- Authorization: token və ya digər kimlik məlumatı.
- Cache-Control: cavabın cache olunması qaydaları.

## 3. HTTP metodları: GET, POST, PUT, PATCH, DELETE

Müsahibədə demək olar ki, mütləq soruşulur.

### GET

- Məlumat almaq üçündür.
- Normal halda serverdə dəyişiklik yaratmamalıdır.
- Idempotent sayılır: eyni GET sorğusunu təkrar etmək nəticəni dəyişməməlidir.

### POST

- Yeni resurs yaratmaq üçün istifadə olunur.
- Adətən body ilə data göndərilir.
- Çox vaxt idempotent deyil. Eyni POST bir neçə dəfə göndərilsə, bir neçə resurs yarana bilər.

### PUT

- Mövcud resursu tam yeniləmək üçün istifadə olunur.
- Əgər tam obyekt göndərirsənsə, PUT daha uyğundur.
- Idempotent-dir.

### PATCH

- Resursun qismən yenilənməsi üçündür.
- Tək bir sahə dəyişəcəksə, PUT əvəzinə PATCH daha məqsədəuyğundur.
- Məsələn yalnız status və ya price dəyişmək.

### DELETE

- Resursu silmək üçündür.
- Idempotent hesab olunur: resurs artıq silinibsə, yenə də eyni məqsədi ifadə edir.

### Müsahibədə güclü cavab nümunəsi:

GET məlumat oxumaq üçündür, POST yaratmaq, PUT tam update, PATCH partial update, DELETE isə silmək üçündür. PUT və DELETE idempotentdir, POST isə adətən deyil.

## 4. Status kodları və real müsahibə nüansları

Sadəcə 200, 404 bilmək kifayət etmir. Məna ilə birlikdə bilməlisən.

- 200 OK - Sorğu uğurlu oldu.
- 201 Created - Yeni resurs yaradıldı.
- 204 No Content - Uğurludur, amma body qaytarmır.
- 400 Bad Request - Client səhv data göndərib.
- 401 Unauthorized - Kimlik təsdiqlənməyib.
- 403 Forbidden - İstifadəçi tanınıb, amma icazəsi yoxdur.
- 404 Not Found - Resurs tapılmadı.
- 409 Conflict - Məsələn duplicate data və ya state konflikti.
- 422 Unprocessable Entity - Format düzgündür, amma biznes qaydasına düşmür.
- 500 Internal Server Error - Server tərəfində gözlənilməz xəta.

### Vacib fərqlər:

- 401 - login/token yoxdur və ya yanlışdır.
- 403 - login var, amma bu əməliyyata səlahiyyət yoxdur.
- 400 - request formatı və ya validation xətası var.
- 404 - endpoint yox, yaxud həmin id-li resurs yoxdur.

## 5. REST prinsipləri və endpoint dizaynı

REST bir protocol deyil, memarlıq üslubudur. Onun əsas prinsipləri bunlardır:

- Client-server ayrılığı
- Stateless - server request-lər arasında client state saxlamır
- Cacheable
- Uniform interface
- Layered system

### Stateless nə deməkdir?

Hər request özündə lazım olan bütün məlumatı daşımalıdır. Server əvvəlki request-i xatırlayıb ona görə qərar verməməlidir. Məsələn auth üçün session-a güvənmək əvəzinə JWT token göndərmək buna nümunədir.

### Yaxşı endpoint necə görünür?

- GET /api/products - bütün məhsullar
- GET /api/products/10 - id-si 10 olan məhsul
- POST /api/products - yeni məhsul yarat
- PUT /api/products/10 - məhsulu tam yenilə
- PATCH /api/products/10 - qismən yenilə
- DELETE /api/products/10 - sil

### Pis nümunələr:

- GET /getAllProducts
- POST /createProduct
- POST /deleteProduct/10

Çünki REST-də fokus action adlarında yox, resursun özündə olmalıdır.

### Naming qaydaları:

- Collection üçün cəm isim istifadə et: /products, /orders
- URL-də verb yazmağa ehtiyac yoxdur.
- Nested relation üçün düşünülmüş dizayn et: /orders/5/items
- Consistency çox vacibdir.

## 6. JSON, DTO, Model Binding, Validation

REST API-lər ən çox JSON ilə işləyir.

### JSON nədir?

Yüngül data formatıdır. Açıq-aşkar açar-dəyər prinsipi ilə işləyir. Frontend və backend arasında məlumat ötürmək üçün çox uyğundur.

Məsələn: {"name":"Mouse","price":25,"stock":40}

### DTO nədir?

DTO - Data Transfer Object. API-yə gələn və ya API-dən çıxan data modelidir. Database entity-ni birbaşa çıxarmaq əvəzinə DTO istifadə etmək daha təhlükəsiz və daha təmiz yanaşmadır.

- CreateProductDto - yaratmaq üçün gələn data
- UpdateProductDto - update üçün gələn data
- ProductResponseDto - client-ə qaytardığın data

### Niyə entity-ni birbaşa qaytarmamaq daha yaxşıdır?

- Lazımsız sahələr expose olunmur.
- Security baxımından safer-dir.
- API contract daha stabil qalır.
- Database modeli dəyişsə belə API-ni qorumaq olur.

### Model Binding nədir?

ASP.NET Core request-dən gələn datanı avtomatik olaraq action parametrinə map edə bilir. Məsələn route-dan id-ni, query-dən page-i, body-dən DTO-nu bağlayır.

- [FromRoute] - route parametrləri
- [FromQuery] - query string
- [FromBody] - request body
- [FromHeader] - headers

### Validation

API-yə gələn datanı yoxlamaq lazımdır. Məsələn name boş olmamalıdır, price mənfi ola bilməz. ASP.NET Core-da Data Annotations və ya FluentValidation istifadə oluna bilər.

- [Required]
- [MaxLength(100)]
- [Range(0.01, 100000)]

#### Yaxşı yanaşma:

Validation controller içində əl ilə if-lərlə yox, ayrı qayda və ya attribute-lərlə edilməlidir. Beləliklə kod təmiz və təkrar istifadə edilən olur.

## 7. ASP.NET Core Web API-də əsas anlayışlar

Müsahibədə .NET tərəfini REST API ilə birləşdirə bilməyin önəmlidir.

### Əsas komponentlər:

- Controller və ya Minimal API
- Routing
- Dependency Injection
- Service layer
- Repository pattern - hər layihədə şərt deyil, amma tez-tez soruşulur
- Entity Framework Core
- Middleware
- Swagger / OpenAPI

### Controller nə edir?

Request qəbul edir, service-ə ötürür, nəticəni uyğun HTTP response kimi qaytarır. Biznes məntiqi controller-də yığılmamalıdır.

### Service layer nə edir?

Biznes qaydalarını daşıyır. Məsələn məhsul yaratmadan əvvəl duplicate olub-olmadığını yoxlayır, stok qaydasını tətbiq edir və s.

### Repository nə üçündür?

Database ilə işləmə təbəqəsini ayırmaq üçün istifadə olunur. Amma hər yerdə məcburi deyil. Bəzi layihələrdə EF Core DbContext birbaşa service-də istifadə olunur.

### Dependency Injection nədir?

Class-ların ehtiyac duyduğu obyektləri öz içində new etməməsi, kənardan verilməsi prinsipi. Bu test etməni və loose coupling-i asanlaşdırır.

- builder.Services.AddScoped<IProductService, ProductService>();
- Controller constructor-da IProductService qəbul edir.

### Middleware nədir?

- Request pipeline-da işləyən komponentlərdir.
- Exception handling, authentication, logging, CORS kimi işlər üçün istifadə olunur.

## 8. Authentication və Authorization

Bu iki termin çox qarışdırılır.

- Authentication - sən kimsən?
- Authorization - sən nə edə bilərsən?

### Məsələn:

- Login olub token almaq - authentication
- Yalnız admin endpoint-ə girmək - authorization

### JWT barədə nə bilməlisən?

- Stateless auth üçün geniş istifadə olunur.

- Client token-i hər request-də Authorization header ilə göndərir.
- Server token-i validate edir.
- Claim-lər istifadəçi haqqında rol və s. məlumat daşıya bilər.

### Role-based authorization

- [Authorize(Roles = "Admin")]
- Bəzi endpoint-lər sadəcə authenticated user, bəziləri isə yalnız müəyyən rol üçün ola bilər.

## 9. Versioning, pagination, filtering, sorting

Real API-lərdə bunları bilmək səni daha yetkin göstərir.

### Versioning

- API dəyişdikcə köhnə client-ləri sındırmamaq üçün versiyalama edilir.
- Məsələn: /api/v1/products
- Header və query ilə versioning də mümkündür.

### Pagination

- Minlərlə data-nı bir dəfəyə qaytarmaq düzgün deyil.
- Məsələn: GET /api/products?page=1&pageSize=20
- Response-da totalCount, page, pageSize kimi metadata qaytarmaq faydalıdır.

### Filtering

- GET /api/products?minPrice=10&maxPrice=100
- GET /api/products?category=electronics

### Sorting

- GET /api/products?sortBy=price&sortOrder=asc
- Whitelist yanaşması ilə yalnız icazə verilən field-lərə sort etmək daha təhlükəsizdir.

## 10. Error handling, logging, Swagger, testing

### Error handling

- Try-catch hər action-da yazmaq yaxşı yanaşma deyil.
- Global exception middleware və ya exception handler daha düzgün həll sayılır.
- Client-ə standard error response formatı qaytarmaq yaxşıdır.

### Logging

- Xəta baş verəndə səbəbi anlamaq üçün vacibdir.
- Information, Warning, Error, Critical səviyyələri var.
- Sensitive data-nı log-a yazmamaq lazımdır.

### Swagger / OpenAPI

- API endpoint-lərini sənədləşdirmək və test etmək üçün istifadə olunur.
- Müsahibədə desən ki, Swagger həm developer rahatlığı, həm də API contract üçün faydalıdır, yaxşı təsir bağışlayar.

### Testing

- Unit test - service və biznes məntiqi test olunur.
- Integration test - endpoint-dən database və digər hissələrə qədər axın test olunur.
- Postman və ya Swagger manual test üçün faydalıdır, amma automated test-i əvəz etmir.

## 11. Təhlükəsizlik və performans

### Təhlükəsizlik

- Input validation olmadan body-ni qəbul etmə.
- SQL Injection-dan qorunmaq üçün parametrli query və ya ORM istifadə et.
- Sensitive məlumatları response-da qaytarma.
- HTTPS istifadə et.
- CORS-u düzgün konfiqurasiya et.
- Rate limiting və throttling bəzi sistemlərdə vacibdir.

### Performans

- AsNoTracking oxuma sorğularında faydalı ola bilər.
- Pagination istifadə et.
- N+1 query problemini tanı.
- Async/await I/O-bound əməliyyatlarda vacibdir.
- Caching bəzi endpoint-lərdə response vaxtını azalda bilər.

### Async/await niyə vacibdir?

Database, file, network kimi I/O əməliyyatlarında thread-i boş yerə bloklamamaq üçün istifadə olunur. ASP.NET Core-da scalability baxımından önəmlidir.

## 12. Tez-tez verilən müsahibə sualları

#### REST ilə SOAP fərqi nədir?

REST daha yüngül və çox vaxt JSON əsaslıdır. SOAP isə daha sərt kontraktlı, XML əsaslı və daha ağır protokoldur.

#### PUT ilə PATCH fərqi nədir?

PUT tam update, PATCH partial update üçündür.

#### Idempotent nə deməkdir?

Eyni request-i neçə dəfə göndərsən də sistemin nəticəsi dəyişmirsə, idempotent sayılır.

#### DTO niyə lazımdır?

Entity-ni gizlətmək, contract-ı qorumaq, təhlükəsizlik və çeviklik üçün.

#### 401 ilə 403 fərqi nədir?

401 tanınmayan istifadəçi, 403 tanınan amma icazəsiz istifadəçi deməkdir.

#### Controller-də biznes məntiqi yazmaq düzgündürmü?

Yox, controller mümkün qədər nazik olmalı, əsas məntiq service qatında olmalıdır.

### Swagger nə üçündür?

API-ni sənədləşdirmək, yoxlamaq və endpoint-ləri rahat görmək üçün.

### Model validation necə edilir?

Data annotations, FluentValidation və ya custom validation yanaşmaları ilə.

## 13. Mini praktik nümunə: Product API

Tutaq ki, sadə Product API hazırlayırsan. Burada ideal axın belə ola bilər:

1. Client POST /api/products endpoint-inə JSON göndərir.
2. Controller CreateProductDto qəbul edir.
3. Model validation işləyir.
4. Service duplicate və biznes qaydalarını yoxlayır.
5. Entity database-ə yazılır.
6. Response olaraq 201 Created qaytarılır.
7. Location header ilə yeni resursun ünvanı göstərilə bilər.

### Məsələn create request body:

{ "name": "Mechanical Keyboard", "price": 120, "stock": 15 }

### Uyğun response:

201 Created

### Bu nümunədə nələri vurğulaya bilərsən?

- DTO istifadə etdim.
- Validation əlavə etdim.
- Controller biznes məntiqi saxlamır.
- Service layer istifadə etdim.
- Uğurlu yaradılmada 201 Created qaytardım.
- Database modeli ilə response modelini ayırdım.

## 14. Müsahibədə necə danışmaq lazımdır?

- Tərifi əzbər demə, məntiqini başa sal.
- Sadə nümunə ver: products, orders, users.
- HTTP metodları və status kodlarını konkret use-case ilə de.
- .NET tərəfdə controller, service, DI, EF Core, validation, swagger kimi anlayışları birləşdir.
- Bilmediyin yerdə uydurma. Məntiqi şəkildə düşünərək cavab ver.

### Yaxşı cavab üslubu nümunəsi:

REST API-ni belə düşünürəm: frontend və backend arasında HTTP üzərindən ünsiyyətdir. Resurs mərkəzlidir, yəni products, orders kimi endpoint-lər qururuq. GET oxumaq, POST yaratmaq, PUT tam update, PATCH partial update, DELETE silmək üçündür. ASP.NET Core-da bunun üçün controller, DTO, validation, service layer və EF Core istifadə oluna bilər.

## 15. 1 günlük hazırlıq planı

8. HTTP metodları və status kodlarını əzbər yox, məntiqlə təkrarla.
9. REST prinsiplərini öyrən: stateless, resource-based URL, uniform interface.
10. .NET Web API-də Controller, DI, service, DTO, validation mövzularını təkrarla.
11. JWT, authentication, authorization fərqini oturt.
12. Swagger və Postman ilə bir demo API yoxla.
13. Özün üçün 5-6 müsahibə sualına səsli cavab məşq et.

## 16. 1 həftəlik hazırlıq planı

14. 1-ci gün: HTTP, status kodları, REST əsasları
15. 2-ci gün: ASP.NET Core controller, routing, model binding
16. 3-cü gün: DTO, validation, service layer, DI
17. 4-cü gün: EF Core CRUD, async/await, pagination
18. 5-ci gün: JWT auth, role authorization, middleware
19. 6-cı gün: Swagger, logging, error handling, testing
20. 7-ci gün: Mini project qur və mock interview et

## 17. Son xülasə

Müsahibə üçün minimum bilməli olduqların bunlardır: REST nədir, HTTP metodları, status kodları, resurs yönümlü endpoint dizaynı, JSON və DTO, validation, ASP.NET Core controller və service anlayışı, JWT auth, error handling, Swagger, async/await və əsas təhlükəsizlik qaydaları.

Əgər bunları sadə dildə nümunə ilə izah edə bilsən, .NET intern səviyyəsi üçün çox güclü görünərsən.

### Qısa yadda saxla bloku

- REST = resource-based HTTP API
- GET oxuyur, POST yaradır, PUT tam yeniləyir, PATCH qismən yeniləyir, DELETE silir
- 200, 201, 204, 400, 401, 403, 404, 500 ən vacib statuslardır
- DTO, validation, service layer, DI - .NET müsahibəsində çox soruşulur
- Auth və authorization fərqini qarışdırma
- Swagger, logging, testing və security barədə də qısa danışa bil